Does Multi-Agent Collaboration Really Help LLM Bug Fixing? A Budget-Controlled, Contamination-Aware Study

Seif Zaki
Zewail City of Science and Technology
ID: 202201554

Karim Wael
Zewail City of Science and Technology
ID: 202202212

Ahmed Ayman
Zewail City of Science and Technology
ID: 202200168

Abstract—Multi-agent LLM pipelines, in which specialised agents localise, patch and review bugs, report strong results in automated program repair (APR): UniDebugger reports fixing every QuixBugs bug, and MAGIS resolves eight times more GitHub issues than direct use of GPT-4. However, Tran and Kiela recently showed that on multi-hop reasoning a single agent matches or outperforms multi-agent systems once the thinkingtoken budget is equal, suggesting that part of the multi-agent advantage is extra compute. Whether this holds for code repair, where agents receive feedback from executing tests, is open. We propose a controlled study and prototype that compares a single-shot LLM, a single-agent iterative loop and a LangGraph multi-agent pipeline (localiser, fixer, test-runner tool, reviewer) under identical per-bug budgets of LLM calls and output tokens. On QuixBugs-Python and a ConDefects-Python sample we measure correct fix rate on held-out tests, plausible-but-incorrect patches, a hallucination taxonomy, cost, and memorisation using identifier-renamed programs. The prototype is exposed as a

FastAPI service with a Streamlit dashboard.

Index Terms—automated program repair, large language models, multi-agent systems, hallucination, data contamination

I. Introduction and Problem Definition

LLM-based program repair has moved from single prompts to agents, and from single agents to teams of agents. MAGIS [2] assigns four roles (Manager, Repository Custodian, Developer, QA Engineer) to resolve GitHub issues and reports an eight-fold higher resolved ratio than direct GPT-4 on SWE-bench. UniDebugger [1] organises agents hierarchically, following cognitive debugging theory, and reports fixing all QuixBugs bugs.

Whether the *team* is what helps is less clear. Tran and Kiela [3] observe that most multi-agent evaluations compare against a single agent that uses much less computation. When thinking-token budgets are equal, a single agent matched or outperformed sequential, debate, role-based and ensemble multi-agent systems on multi-hop reasoning. Agentless [4] points the same way in software engineering: a fixed pipeline without an agent loop was competitive with agent-based systems. Tran and Kiela studied text reasoning only; in repair, agents also receive test-execution feedback, and long failure logs may degrade a single agent's context, the condition under which they expect multi-agent systems to regain an advantage.

Two further confounds affect repair results. Patches that pass the available tests (*plausible*) are often not correct [5]. And QuixBugs [9] is public and saturated: o1 fixes 40/40 of its Python bugs [11], so it can neither show improvement nor separate repair ability from memorisation.

Problem statement. Does multi-agent collaboration improve LLM-based bug fixing once budget, patch correctness and contamination are controlled?

Planned contributions. (a) An equal-budget comparison of single-shot, single-agent and multi-agent repair with the same model; (b) correctness on held-out tests and a hallucination taxonomy for repair agents; (c) a memorisation probe using semantics-preserving identifier renaming; (d) an open prototype (FastAPI, LangGraph, Streamlit).

II. Literature Review

Our review centres on three works: the closest multiagent repair system, the multi-agent design whose ablation we adapt, and the equal-budget evidence that motivates our main question (Table I).

UniDebugger [1] (EMNLP 2025). *Problem:* fault localisation and repair are usually solved by separate tools. *Method:* a hierarchical multi-agent framework whose coordination follows cognitive debugging theory; lower levels localise and fix, and more agents are engaged for harder bugs. *Results:* it reports fixing all QuixBugs bugs and more Defects4J bugs than prior repair methods. *Relevance:* it is the closest system to ours and uses our primary dataset, so it is our main point of comparison. *Open for us:* whether its gain persists against a single agent given the same budget.

MAGIS [2] (NeurIPS 2024). *Problem:* resolving repository-level GitHub issues. *Method:* four role-based agents split the work into planning and coding; a QA Engineer reviews the Developer's changes. *Results:* 13.94% of SWE-bench issues resolved, eight times direct GPT-4; without the QA Engineer 10.63%, without hints 10.28%, without both 8.71%. *Relevance:* its ablation shows that removing the reviewer costs about a quarter of the fixes, which motivates our reviewer ablation. *Open for us:* its baseline is a single direct GPT-4 call, not a single agent with the same budget, and its tasks are repository-level rather than function-level bugs checked against held-out tests.

Tran and Kiela [3] (arXiv 2026). *Problem:* are multiagent gains due to architecture or to extra compute? *Method:*

TABLE I Core related work and our position

Work	Task / data	Key result	What we add
UniDebugger [L1]ocalisation	+ repair; QuixBugs, Defects4J	All QuixBugs bugs fixed	Single agent with equal budget; held-out tests
MAGIS [2]	Repo-level issues; SWE-bench	13.94%; 10.63% without QA agent	Reviewer ablation on function-level bugs vs. held-out tests
Tran & Kiela [3]	Multi-hop QA	Single ≥ multi at equal tokens	Same question for repair with test execution
This work	Functionlevel repair; QuixBugs, ConDefects	–	Equal budget, held-out tests, renaming probe

a single agent is compared with several multi-agent designs at matched thinking-token budgets, supported by an informationtheoretic argument based on the Data Processing Inequality. *Results:* the single agent matches or outperforms multi-agent systems; multi-agent systems become competitive when the single agent's context is degraded or more compute is spent. *Relevance:* it defines our equal-budget design and gives a testable hypothesis for repair. *Caveat:* it is a preprint and covers text reasoning, not code execution.

Supporting work. Qi et al. [5] show that most patches of earlier repair tools were plausible but incorrect, which defines our correctness metric. Self-Debugging [6] shows that execution feedback helps LLMs fix their own code; our singleagent baseline follows it. LLMs struggle to self-correct without external feedback [7], so a reviewer agent may approve wrong patches. MAST [8] offers a taxonomy of multi-agent failures that we use to classify ours. ConDefects [10] provides 1,625 Python faulty programs from 2021–2023 built to reduce training-data leakage.

III. Research Gap and Questions

Multi-agent repair systems report gains over prior methods [1], [2], while the equal-budget evidence against multiagent systems comes from text reasoning [3]. We are not aware of an equal-budget single- versus multi-agent comparison for program repair with test execution that also checks patches against held-out tests. We ask:

RQ1 (Equal budget). With the same per-bug budget of LLM calls and output tokens, does a multi-agent pipeline fix more bugs than a single-agent iterative loop? *Hypothesis [3]:* no on short programs; possibly yes when long test logs degrade the single agent's context. *Metric:* correct fix rate, paired per bug, analysed against prompt length.

RQ2 (Reviewer). Does a reviewer agent reduce plausiblebut-incorrect patches, as MAGIS's ablation suggests [2], or does it approve them [7]? *Metrics:* overfitting rate, reviewer precision/recall against held-out test outcomes, hallucination counts.

RQ3 (Contamination). How much does success drop on semantics-preserving, identifier-renamed versions of the same programs, and does the drop differ between QuixBugs and ConDefects? *Metric:* change in correct fix rate.

IV. Proposed Approach and Evaluation Plan

Datasets. (i) QuixBugs-Python: 40 programs with a one-line bug, tests and a reference fix [9]. (ii) About 100 ConDefectsPython bugs sampled by difficulty [10]. (iii) Renamed variants of (i) and (ii), produced with Python's ast module and checked by running the reference fix. Each bug's tests are split into *visible* tests (given to the systems) and *held-out* tests (used only for evaluation).

Systems. All use the same LLM and temperature and stop at K LLM calls or T output tokens per bug (e.g., $K = 8$), whichever comes first.

- S0, single-shot: one prompt, one patch.
- S1, single agent: patch, run visible tests, return failures to the model, repeat until pass or budget exhausted [6].
- S2, multi-agent (LangGraph): Localiser (LLM) → Fixer (LLM) → Test-runner (sandboxed tool, not an LLM) → Reviewer (LLM), looping to the Fixer on failure or rejection.
- S2−R: S2 without the Reviewer (ablation for RQ2).

Evaluation module. Runs each patch in a sandboxed subprocess with a timeout against visible and held-out tests and labels it *correct*, *plausible-but-incorrect* or *failing*. It flags hallucinations: (H1) non-runnable code; (H2) non-existent API; (H3) edits outside the buggy function or to tests; (H4) unchanged code reported as fixed; (H5) reviewer approval of a patch that fails held-out tests. Multi-agent failures are also mapped to MAST categories [8]. Two members compare a random 20% of plausible patches with the reference fix and report Cohen's κ.

Analysis. Three runs per bug and condition; means with 95% confidence intervals; McNemar's test on paired per-bug correctness (S1 vs. S2, S2 vs. S2−R).

Prototype. FastAPI POST /repair taking {code, tests, mode} and returning {patch, verdict, agent trace, tokens}; LangGraph orchestration; a Streamlit dashboard for fix rates, hallucinations, cost and per-bug traces.

Threats and expected SOTA answer. Contamination cannot be removed entirely; RQ3 measures part of it. QuixBugs has 40 bugs, so intervals will be wide. Since UniDebugger and o1 already fix all QuixBugs bugs, we do not expect to beat the state of the art in raw fix count; our contribution is the controlled comparison and the correctness and contamination analysis.

V. Timeline and Work Split

Wk 7: proposal. Wk 8: dataset loaders, sandbox, S0/S1. Wk 9: S2 in LangGraph, evaluation module, FastAPI. Wk 10: early QuixBugs results (Phase 2). Wk 11–12: ConDefects, renaming probe, manual labelling. Wk 13: dashboard and statistics. Wk 14: paper and demo. *Seif Zaki:* datasets and evaluation module. *Karim Wael:* agents and FastAPI. *Kareem Ezzaldin:* dashboard, labelling and statistics. All members write the report and commit their own work.

AI Tools Declaration

Claude Code (Anthropic) was used to search for and verify literature and to draft this proposal. All content was reviewed and edited by the authors.

References

[1] C. Lee, C. S. Xia, L. Yang, J. Huang, Z. Zhu, L. Zhang, and M. R. Lyu, "UniDebugger: Hierarchical multi-agent framework for unified software debugging," in *Proc. EMNLP*, 2025, arXiv:2404.17153.

[2] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng, "MAGIS: LLM-based multi-agent framework for GitHub issue resolution," in *Proc. NeurIPS*, 2024.

[3] D. Tran and D. Kiela, "Single-agent LLMs outperform multi-agent systems on multi-hop reasoning under equal thinking token budgets," arXiv:2604.02460, 2026.

[4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," in *Proc. FSE*, 2025, arXiv:2407.01489.

[5] Z. Qi, F. Long, S. Achour, and M. Rinard, "An analysis of patch plausibility and correctness for generate-and-validate patch generation systems," in *Proc. ISSTA*, 2015.

[6] X. Chen, M. Lin, N. Scharli, and D. Zhou, "Teaching large language¨ models to self-debug," in *Proc. ICLR*, 2024.

[7] J. Huang *et al.*, "Large language models cannot self-correct reasoning yet," in *Proc. ICLR*, 2024.

[8] M. Cemri, M. Pan *et al.*, "Why do multi-agent LLM systems fail?" in *Proc. NeurIPS*, 2025, arXiv:2503.13657.

[9] D. Lin, J. Koppel, A. Chen, and A. Solar-Lezama, "QuixBugs: A multilingual program repair benchmark set based on the Quixey challenge," in *Proc. SPLASH Companion*, 2017.

[10] Y. Wu *et al.*, "ConDefects: A complementary dataset to address the data leakage concern for LLM-based fault localization and program repair," in *Proc. FSE Companion (Demo)*, 2024, arXiv:2310.16253.

[11] H. Hu, Y. Shang, G. Xu, C. He, and Q. Zhang, "Can GPT-O1 kill all bugs? An evaluation of GPT-family LLMs on QuixBugs," in *Proc. Int. Workshop on Automated Program Repair (APR)*, 2025, arXiv:2409.10033.